Bioscara

Sebastian Storz

Jan 30, 2026

CONTENTS:

CHAPTER

ONE

OVERVIEW

TODO:

Note

A project overview should briefly describe the scope, purpose, and goals of the project. It helps users quickly understand what the project is about and why it exists. When writing an overview, mention the main problem the project solves, its target audience, and any unique features or technologies it uses.

Example:

> *This project provides a stadaridized template for DALSA GitHub repositories, designed to help project owners quickly set up new projects with a consistent structure and DevOps features. Its goal is to ensure maintainability and collaboration by enforcing standards that make future contributions and usage seamless. The template integrates Sphinx + Doxygen for unified documentation of both Python and C++ packages, and uses GitHub Actions to automate generation of web-based documentation, code analyis (linting), code rewrites to follow a consistent style (formatting), and execution of tests.*

Here you will find a full description of the project's architecture and core concepts, detailed documentation of all packages, and practical usage examples. This guide is designed to help you understand the structure and functionality of the project, making it easier to get started and make effective use of its features.

TODO:

1.1 Documentation

The C++ API documentation is parsed by doxygen and can be found here.

1.1.1 bioscara_hardware_interfaces

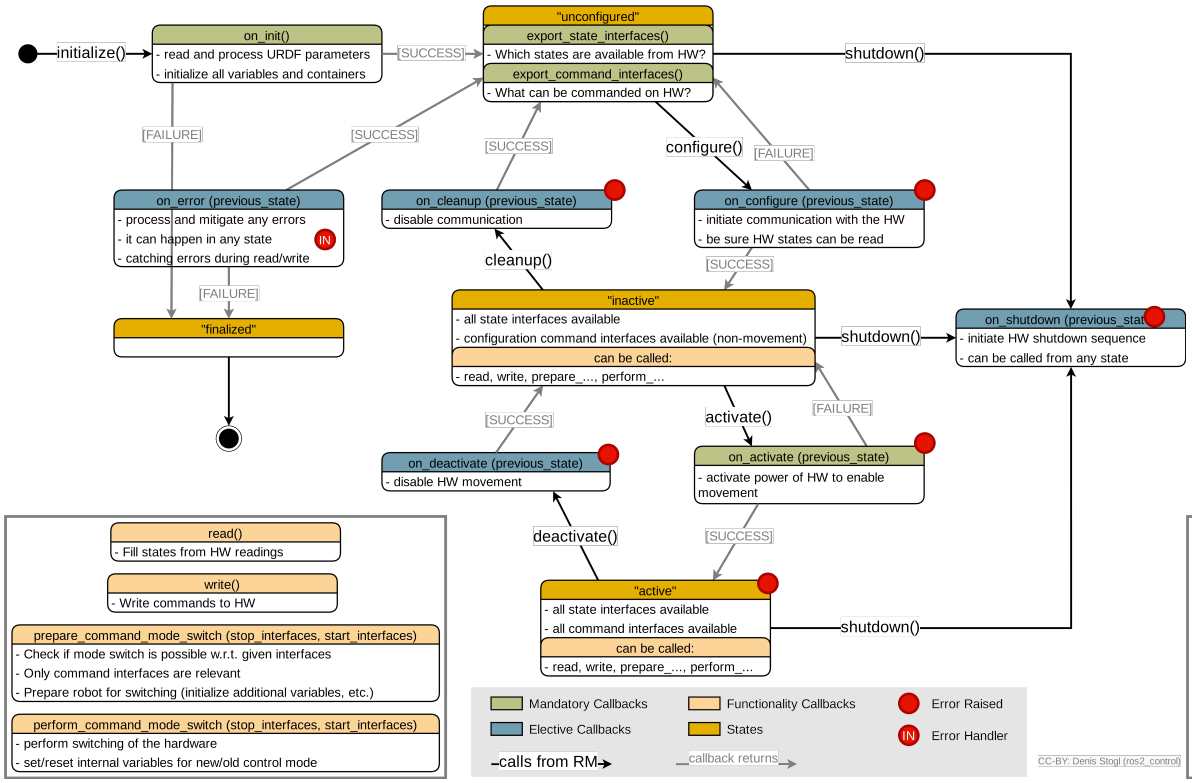
constexpr char `bioscara_hardware_interfaces::HW_IF_HOME`[] = "home"

class `BioscaraArmHardwareInterface` : public hardware_interface::SystemInterface

#include <arm_hardware.hpp> The bioscara arm hardware interface class.

The hardware interface serves to wrap custom hardware interaction with the arm joints in the standardized ros2_control architecture.

Hardware Lifecycle The hardware follows the ros2_control hardware interface lifecyle which intern is following the ROS2 managed node lifecycle.



Public Functions

hardware_interface::CallbackReturn **on_init**(const hardware_interface::HardwareComponentInterfaceParams &params) override

Called on initialization to the `unconfigured` state.

Performs the following checks on the configures joints parsed form the URDF description:

- Each joint must have the 3 command interfaces (in this order): ‘position’, ‘velocity’, ‘home’
- Each joint must have the 3 state interfaces (in this order): ‘position’, ‘velocity’, ‘home’

Stores the configuration parameters for each joint in the *_joint_cfg* map. Each joint must have these parameters:

- i2c_address (int, HEX)
- reduction (float)
- min (float)
- max (float)
- stall_threshold (int, DEC)
- hold_current (int, DEC)
- drive_current (int, DEC)
- max_acceleration (float)
- max_velocity (float)
- homing

- speed (float)
- threshold (int, DEC)
- current (int, DEC)
- acceleration (float)

Adds each joint to the internal _joints map. Creates a MockJoint object if the use_mock_hardware parameter is 'True' or 'true', or else a hardware Joint.

Parameters
params –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_shutdown**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive`, `unconfigured` and `active` to the `finalized` state.

When transitioning directly from `active` to `finalized` *on_deactivate()* is automatically called before Source Code If the previous state is either `inactive` or `active` the *on_cleanup()* method is called first. Then regardless of the previous state, the _joints map is cleared.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_configure**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `unconfigured` to the `inactive` state.

Establish and test connection to each joint.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_cleanup**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive` to the `unconfigured` state.

Disconnect from the joints.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_activate**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive` to the `active` state.

Calls *activate_joint()* to enable the joints.

It is allowed to activate the hardware even if it is not homed. To home the joint the homing_controller must be activated, but generally a hardware component must be active in order for controllers to become active.

To prohibit movement on activation the set point for each position command interface is set equal to the current measured position, and the velocity command is set to 0.0 for each command interface. The current values are obtained by calling the *read()* method once which populates the state interfaces with values.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_deactivate**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `active` to the `inactive` state.

Disables all joints and thereby allows backdriving. State interfaces continue to be updated.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::return_type **read**(const rclcpp::Time &time, const rclcpp::Duration &period) override

Reads from the hardware and populates the state interfaces.

Iterates over all state interfaces and calls the corresponding Joint method.

- State interface "position" -> *bioscara_hardware_drivers::Joint::getPosition()*
- State interface "velocity" -> *bioscara_hardware_drivers::Joint::getVelocity()*
- State interface "home" -> *bioscara_hardware_drivers::Joint::isHomed()*
  - This does not actually trigger a communication, instead it relies on the return flags of the previous transmissions. Since position and velocity have been called immediatly before the return flags are assumed to be valid.
  - If the the homing of a joint has been activated through the command interface (*bioscara_hardware_drivers::Joint::getCurrentBCmd()* == *bioscara_hardware_drivers::Joint::HOME*) the device signals BUSY (*bioscara_hardware_drivers::Joint::isBusy()*) as long as it is still homing.

    If the BUSY flag is reset while the current command is still *bioscara_hardware_drivers::Joint::HOME* we can assume the homing has finished. Then the "home" command interface of the joint is reset to 0.0, which will stop the homing (perform cleanup tasks) at the next write cycle.

Parameters
- **time** –
- **period** –

Returns
hardware_interface::return_type

hardware_interface::return_type **write**(const rclcpp::Time &time, const rclcpp::Duration &period) override

Writes commands to the hardware from the command interfaces.

In contrast to the *read()* method the *write()* method only loops over the command interfaces that are currently active defined by the *_joint_command_modes* map. See *prepare_command_mode_switch()* for a detailed reasoning why this approach has been chosen.

- Command interface "position" -> *bioscara_hardware_drivers::Joint::setPosition()*

- Command interface "velocity" -> *bioscara_hardware_drivers::Joint::setVelocity()*
- Command interface "home" -> *bioscara_hardware_drivers::Joint::startHoming()*
  - If the commanded value in "home" is != 0.0 the and the joint is currently executing a blocking function, for example homing (*bioscara_hardware_drivers::Joint::getCurrentBCmd()* == *bioscara_hardware_drivers::Joint::NONE*), the homing sequence is started with the speed, sensitivity, current and acceleration defined in the *_joint_cfg* which is polulated from the hardware description urdf. The direction of the homing is determined by the sign of the command interface value.
  - If the commanded value in "home" is = 0.0 and the joint is currently executing homing, the homing is stopped. This can either happen prematurely through user input or when the homing is completed which is registered in *read()*.

Parameters

- **time** –
- **period** –

Returns
hardware_interface::return_type

hardware_interface::return_type **prepare_command_mode_switch**(const std::vector<std::string> &start_interfaces, const std::vector<std::string> &stop_interfaces) override

Performs checks and book keeping of the active control mode when changing controllers in non-realtime context.

For safe operation only one controller may interact with the hardware at the time. For example if the velocity JTC is active and has claimed the velocity command interfaces it is technically possible to activate the position JTC (or a homing controller, or others) that claim a different command interface (position in this case). However if both controllers are active they start writing to the hardware simultaneously which is to be avoided. For this reason a book keeping mechanism has been implemented which stores the currently active command interfaces for each joint in the *_joint_command_modes* member. Each joint has a set of active command interfaces. When a controller switch is performed the interfaces that should be stopped are removed from each joint set, then the one that should be started are added, if they are already present an error is thrown. Lastly a validation is performed. Currently the validation is simple since each joint may only have one command interface. The validation can be expanded for furture use cases that require a combination of active command interfaces per joint for example.

The following basic checks are implemented:

- **On deactivation**:
  - [ERROR] Homing command interfaces may only be deactivated if no current homing process is ongoing (*bioscara_hardware_drivers::Joint::getCurrentBCmd()* != *bioscara_hardware_drivers::Joint::HOME*)
  - [WARN] Deactivating a velocity command interface if the velocity set point is 0.0.
  - [WARN] Deactivating a command interface that has not been started. This should not happen.
- **On activation**:
  - [ERROR] Activating a command interface that is already started. This should not happen.
  - [ERROR] Activating a second command interface for a joint.

- [ERROR] Activating 'position' or 'velocity' command interface if the joint is not homed (*bioscara_hardware_drivers::Joint::isHomed()* == false).

Since this method operates in non-realtime context it must not access critical members (*_joint_command_modes* and *_joints*) to avoid priority inversion. Therefore the new command modes are first saved to a cache *_new_joint_command_modes*. This will then be applied to *_joint_command_modes* in the *perform_command_mode_switch()* which is executed in RT context.

Parameters

- **start_interfaces** – command interfaces that should be started in the form "joint/interface"
- **stop_interfaces** – command interfaces that should be stopped in the form "joint/interface"

Returns
hardware_interface::return_type

hardware_interface::return_type **perform_command_mode_switch**(const std::vector<std::string> &start_interfaces, const std::vector<std::string> &stop_interfaces) override

Perform the mode-switching for the new command interface combination in realtime context.

Performs the following actions:

- acquires the *mtx* lock to protect the critical members.
- Copies the cached *_new_joint_command_modes* to the *_joint_command_modes*
- **On activation**:
  - **home** interface:
    * Reset command to 0.0. This clears any remaining commands that have been written to the command interface while the hardware was unable to act on it. For example if it was inactive or the homing command was not the active command mode.

Parameters

- **start_interfaces** – vector of string identifiers for the command interfaces starting.
- **stop_interfaces** – vector of string identifiers for the command interfaces stopping.

Returns
return_type::OK if the new command interface combination can be switched to (or) if the interface key is not relevant to this system. Returns return_type::ERROR otherwise.

hardware_interface::CallbackReturn **on_error**(const rclcpp_lifecycle::State &previous_state) override

Called when an error in any state or state transition is thrown.

According to the ros2_control documentation:

> Error handling follows the node lifecycle. If successful CallbackReturn::SUCCESS is returned and hardware is again in `UNCONFIGURED` state, if any ERROR or FAILURE happens the hardware ends in `FINALIZED` state and can not be recovered. The only option is to reload the complete plugin, but there is currently no service for this in the Controller Manager.

Since the hardware will immediatly return to the `unconfigured` state (source) if the error could be handled we manually call the transition functions which would normally be called to this state. Those are:

- **Previous state**: `active`
  - Deactivate hardware (*on_deactivate()*) -> `inactive`
  - Clean-Up hardware (*on_cleanup()*) -> `unconfigured`
- **Previous state**: `inactive`
  - Deactivate hardware (*on_deactivate()*) -> `inactive`
    - call the deactivate function anyway regardless if state was active or inactive. For example if the *on_activate()* function fails on *bioscara_hardware_drivers::Joint::enableStallguard()* the joint will have been enabled, to disable it invoke *on_deactivate()*.
  - Clean-Up hardware (*on_cleanup()*) -> `unconfigured`

In particular the deactivation is important. For example if a joint stalls the *read()* or *write()* methods throw an error, which will be handled here and allow the hardware to be deactivated, disableing the joints to allow backdriving.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

Private Functions

bioscara_hardware_drivers::*err_type_t* **start_homing**(const std::string name, float velocity)

wrapper method to start homing.

Activate the joint, set homing acceleration and start homing.

Parameters

- **name** –
- **velocity** –

Returns
bioscara_hardware_drivers::err_type_t

bioscara_hardware_drivers::*err_type_t* **stop_homing**(const std::string name)

wrapper method to stop homing.

Stop the homing. Reset acceleration and velocity and perform the postHoming cleanup, then deactivate the joint.

Parameters
name –

Returns
bioscara_hardware_drivers::err_type_t

void **split_interface_string_to_joint_and_name**(std::string interface, std::string &joint_name, std::string &interface_name)

Split a interface string like "<joint_name>/<interface_name>" to "<joint_name>" and "<interface_name>".

Parameters

- **interface** –
- **joint_name** –

- **interface_name** –

bioscara_hardware_drivers::*err_type_t* **activate_joint**(const std::string name)

Enables each joint, enables the stall detection and sets the maximmum acceleration.

Parameters
name – joint name to enable

Returns
bioscara_hardware_drivers::err_type_t

bioscara_hardware_drivers::*err_type_t* **deactivate_joint**(const std::string name)

Disables each joint.

Parameters
name – joint name to disable

Returns
bioscara_hardware_drivers::err_type_t

Private Members

std::unordered_map<std::string, std::unique_ptr<bioscara_hardware_drivers::BaseJoint>> **_joints**

unordered map storing the pointers to BaseJoint objects. This will either be a MockJoint or Joint.

An unordered map is chosen to simplify acces via the joint name, as this conforms well with the ROS2_control hardware interface The map does not need to be ordered. Search, insertion, and removal of elements have average constant-time complexity.

Since the BaseJoint methods are implemented as virtual, dynamic method dispatch can be utilized to call the correct implementation of a method. So either BaseJoint::foo() or Joint::foo()/MockJoint::foo() if foo() is overwritten in Joint or MockJoint. a smart pointer is used to guarantee destruction when the pointer is destructed. A unique pointer is used to prevent copying of the object.

std::unordered_map<std::string, *joint_config_t*> **_joint_cfg**

unordered map storing the configuration struct of the joints.

An unordered map is chosen to simplify acces via the joint name, as this conforms well with the ROS2_control hardware interface The map does not need to be ordered. Search, insertion, and removal of elements have average constant-time complexity.

std::unordered_map<std::string, std::set<std::string>> **_joint_command_modes**

unordered map of sets storing the active command interfaces for each joint.

Each joint can have a set of active command interfaces. This type of structure is chosen to group interfaces by joint. In the *write()* function the interface name can simply be constructed by concatenating joint name with interface name. Although currently only one active command interface is allowed at the time, a set can be used to store multiple command interfaces that are acceptable to be combined, for example it would be acceptable to set velocity and driver current and hence that would be an allowable combination.

An unordered map is chosen to simplify acces via the joint name, as this conforms well with the ROS2_control hardware interface. The map does not need to be ordered. Search, insertion, and removal of elements have average constant-time complexity.

std::unordered_map<std::string, std::set<std::string>> **_new_joint_command_modes**

Temporary cache of new joint_command_modes when switching controllers.

Since the *prepare_command_mode_switch()* is executed in a non-RT context we save the new joint command modes to this cache first to avoid needing to lock the *_joint_command_modes*.

std::vector<std::pair<std::string, hardware_interface::InterfaceDescription*>>
_ordered_joint_state_interfaces_ptr

A vector of a pair of interface name and pointer of the hardwares state interface which is ordered by joint and state interface type.

A vector is chosen since it guarantees the correct order by insertion. The vector holds pointers to the actual interface structs stored in the parents HardwareComponentInterface::joint_state_interfaces_ but in the desired order. The order is:

i. position

ii. velocity

iii. home

This order guarantees that when reading the state interfaces in *read()* that the 'home' interface is read last and has the latest state flags from the joint.

std::mutex **mtx**

A mutex that is used to prevent concurrent access to hardware and *_joint_command_modes*.

The mutex prevents two things:

- Modifying the *_joint_command_modes* concurrently.
- Concurrent access to the hardware via the *_joints* map. The Joint harwdare is not thread safe. In particular the *read()* and *write()* methods are executed in one RT thread while the *perform_command_mode_switch()* is called from another RT thread. The latter also tries to modify the Joint object via *activate_joint()* and *deactivate_joint()* which must not happen concurrently with a *read()* or *write()* call.

All methods that need to acquire this lock need to be performed in RT context to avoid being preempted by other lower priority threads potentially blocking the other RT threads from continuing.

struct **joint_config_t**

configuration structure holding the passed paramters from the ros2_control urdf

Saving all parameters on initialization in a structure allows for quick access during runtime.

Public Members

int **i2c_address**

[int, HEX] I2C device adress

1-byte I2C device adress (0x11 … 0x14) for J1 … J4

float **reduction** = 1

[float] Gear reduction of the joint

This is used to transform position and velocity values between in joint units and actuator (stepper) units. The sign depends on the direction the motor is mounted and is turning. Adjust such that the joint moves in the positive direction on on positive joint commands. Cable polarity has no effect since the motors automatically adjust to always run in the 'right' direction from their point of view.

J1: 35

J2: -2*pi/0.004 (4 mm linear movement per stepper revolution, positive rotation makes the joint go down (negative), hence the minus sign for correction)

J3: 24

J4: 12

float **min**

[float] Lower joint limit in joint units

Initial values are (subject to tuning)

J1: -3.04647

J2: -0.0016

J3: -2.62672

J4: -3.01069

float **max**

[float] Upper joint limit in joint units

Initial values are (subject to tuning)

J1: 3.04647

J2: 0.3380

J3: 2.62672

J4: 3.01069

u_int8_t **drive_current**

[int, DEC] Drive current of stepper driver in 0-100 % of 2.5A output (check uStepper doc.)

Set when joint is enabled, see *bioscara_hardware_drivers::BaseJoint::enable()*.

u_int8_t **hold_current**

[int, DEC] Hold current of stepper driver in 0-100 % of 2.5A output (check uStepper doc.)

Set when joint is enabled, see *bioscara_hardware_drivers::BaseJoint::enable()*.

u_int8_t **stall_threshold**

[int, DEC] Stall protection threshold in 0-255, where lower is more sensitive.

If the PID error exceeds the set threshold a stall is triggered and the motor disabled. Set when joint is enabled, see *bioscara_hardware_drivers::BaseJoint::enableStallguard()*.

float **max_velocity**

[float] Maximum permitted joint velocity.

In rad/s or m/s for cylindrical and prismatic joints respectively. Set when joint is enabled, see *bioscara_hardware_drivers::BaseJoint::setMaxVelocity()*.

> **Note**
>
> This is the “last” limit and shall ALWAYS be greater than all limits set in the controller and application (MoveIt) configuration.

float **max_acceleration**

[float] Maximum permitted joint acceleration (and deceleration).

In rad/s^2 or m/s^2 for cylindrical and prismatic joints respectively. Set when joint is enabled, see *bioscara_hardware_drivers::BaseJoint::setMaxAcceleration()*.

> **Note**
>
> This is the “last” limit and shall ALWAYS be greater than all limits set in the controller and application (MoveIt) configuration.

joint_homing_config_t **homing**

Holding the *joint_homing_config_t* configruation values.

struct **joint_homing_config_t**

configuration structure holding the passed homing paramters from the ros2_control urdf

Saving all parameters on initialization in a structure allows for quick access during runtime. See bioscara_hardware_drivers::BaseJoint::_home() for the description of the homing procedure.

Public Members

float **speed**

[float] Signed homing velocity.

In rad/s or m/s for cylindrical and prismatic joints respectively. Must satisfy: `1.0 < RAD2DEG(JOINT2ACTUATOR(<speed>, reduction, 0)) / 6 < 250.0`

u_int8_t **threshold**

[int, DEC] Encoder error threshold 0 to 255.

Lower values make the homing more sensitve but also more susceptible to false positives.

u_int8_t **current**

[int, DEC] Homing current between 0 and 100.

Determines how easy it is to stop the motor and thereby provoke a stall. See *joint_config_t::drive_current*.

float **acceleration** = 0.01

[float] Joint acceleration during homing.

In rad/s^2 or m/s^2 for cylindrical and prismatic joints respectively. Accelerating too fast can trigger a stall and hence false positve homing. See *joint_config_t::max_acceleration*.

class **BioscaraGripperHardwareInterface** : public hardware_interface::SystemInterface

#include <gripper_hardware.hpp> The bioscara gripper hardware interface class.

System interface has been chosen to allow future modifications for grippers that provide feedback. refer to the *BioscaraArmHardwareInterface* class for a more detailed description of the hardware life-cycle.

Public Functions

hardware_interface::CallbackReturn **on_init**(const hardware_interface::HardwareComponentInterfaceParams &params) override

hardware_interface::CallbackReturn **on_shutdown**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive`, `unconfigured` and `active` to the `finalized` state.

When transitioning directly from `active` to `finalized` *on_deactivate()* is automatically called before Source Code If the previous state is either `inactive` or `active` the *on_cleanup()* method is called first.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_configure**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `unconfigured` to the `inactive` state.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_cleanup**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive` to the `unconfigured` state.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_activate**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `inactive` to the `active` state.

Enables PWM generation.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::CallbackReturn **on_deactivate**(const rclcpp_lifecycle::State &previous_state) override

Called on the transistion from the `active` to the `inactive` state.

Disables PWM generation.

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

hardware_interface::return_type **read**(const rclcpp::Time &time, const rclcpp::Duration &period) override

Reads from the hardware and populates the state interfaces.

Invokes the *bioscara_hardware_drivers::Gripper::getPosition()* method to read the gripper position.

Parameters

- **time** –
- **period** –

Returns
hardware_interface::return_type

hardware_interface::return_type **write**(const rclcpp::Time &time, const rclcpp::Duration &period) override

Writes commands to the hardware from the command interfaces.

Invokes the *bioscara_hardware_drivers::Gripper::setPosition()* method to write the command to the gripper.

Parameters

- **time** –
- **period** –

Returns
hardware_interface::return_type

hardware_interface::CallbackReturn **on_error**(const rclcpp_lifecycle::State &previous_state) override

Called when an error in any state or state transition is thrown.

According to the ros2_control documentation:

> Error handling follows the node lifecycle. If successful CallbackReturn::SUCCESS is returned and hardware is again in `UNCONFIGURED` state, if any ERROR or FAILURE happens the hardware ends in `FINALIZED` state and can not be recovered. The only option is to reload the complete plugin, but there is currently no service for this in the Controller Manager.

Since the hardware will immediatly return to the `unconfigured` state (source) if the error could be handled we manually call the transition functions which would normally be called to this state. Those are:

- **Previous state**: `active`
  - Deactivate hardware (*on_deactivate()*) -> `inactive`
  - Clean-Up hardware (*on_cleanup()*) -> `unconfigured`
- **Previous state**: `inactive`
  - Deactivate hardware (*on_deactivate()*) -> `inactive`
    - call the deactivate function anyway regardless if state was active or inactive.
  - Clean-Up hardware (*on_cleanup()*) -> `unconfigured`

Parameters
previous_state –

Returns
hardware_interface::CallbackReturn

Private Members

std::unique_ptr<bioscara_hardware_drivers::BaseGripper> **_gripper**

Smart pointer to the local BaseGripper.

This will be used to either interact with the hardware or mock hardware. A smart pointer is used to guarantee destruction when the pointer is destructed. A unique pointer is used to prevent copying of the object.

gripper_config_t **_gripper_cfg**

configuration struct of the gripper.

float **_last_pos** = std::numeric_limits<double>::quiet_NaN()

float **_vel** = std::numeric_limits<double>::quiet_NaN()

struct **gripper_config_t**

configuration structure holding the passed paramters from the ros2_control urdf

Saving all parameters on initialization in a structure allows for quick access during runtime.

Public Members

float **reduction** = 1

float **offset** = 0

float **min**

float **max**

float **init_pos**

1.1.2 bioscara_hardware_drivers

enum class `bioscara_hardware_drivers::`**err_type_t**

Enum defining common error types.

Values:

enumerator **OK**

Success.

enumerator **ERROR**

Generic Error.

enumerator **NOT_HOMED**

Joint not homed or failed to home.

enumerator **NOT_ENABLED**

Joint not enabled or failed to enable.

enumerator **STALLED**

Joint stalled.

enumerator **NOT_INIT**

Joint not initialized.

enumerator **COMM_ERROR**

Communication error.

enumerator **INVALID_ARGUMENT**

Argument violates conditions.

enumerator **INCORRECT_STATE**

Joint is busy executing another blocking command.

std::string `bioscara_hardware_drivers::`**error_to_string**(*err_type_t* err)

Converts an error code to a string and returns it.

Parameters
err –

Returns
std::string

class **BaseGripper**

#include <mBaseGripper.h> Generic base class for gripper control implementations.

This class is a wrapper function to interact with the robot gripper either through a *MockGripper* or hardware *Gripper* object.

Subclassed by *bioscara_hardware_drivers::Gripper*, *bioscara_hardware_drivers::MockGripper*

Public Functions

BaseGripper(float reduction, float offset, float min, float max, float backup_init_pos)

Construct a new *BaseGripper* object.

The gripper has the reduction r and offset o parameters which are used to translate from a desired gripper width to the servo angle. The relationship between gripper width w and acutator angle α is as follows:

$$\alpha = r(w - o)$$

To determine these parameters execute the following steps:

i. Manually set the gripper to an open position by setting a actuator angle. Be carefull to not exceed the physical limits of the gripper since the actuator is strong enough to break PLA before stalling.

ii. Measure the gripper width w_1 and note the set actuator angle α_1.

iii. Move the gripper to a more closed position that still allows you to accurately measure the width

iv. Measure the second width w_2 and note the corresponding angle α_2

v. Calculate the offset o:

$$o = \frac{\alpha_1 w_2 - \alpha_2 w_1}{\alpha_1 - \alpha_2}$$

vi. Calculate the reduction r:

$$r = \frac{\alpha_1}{w_1 - o}$$

Parameters

- **reduction** – the gripper width to actuator reduction ratio
- **offset** – gripper width to actuator zero offset
- **min** – lower limit in m
- **max** – upper limit in m
- **backup_init_pos** – initial position assumed if none can be retrieved from the buffer file

~BaseGripper(void)

Destroy the *BaseGripper* object.

invokes the *disable()* and *deinit()* method to clean up.

virtual *err_type_t* **init**(void)

Initializes the gripper.

Returns
err_type_t::OK

virtual *err_type_t* **deinit**(void)

Deinitializes the gripper.

Returns
err_type_t::OK

virtual *err_type_t* **enable**(void)

Enables the gripper.

Since the gripper has no position feedback its position is initially unknown. For this reason we try to retrieve the latest commanded position from a buffer file using retrieve_last_position(). If this fails the _backup_init_pos is used.

Returns
err_type_t::OK

virtual *err_type_t* **disable**(void)

Disables the gripper.

As described in *enable()*

Returns
err_type_t::OK

virtual *err_type_t* **setPosition**(float width)

Sets the gripper width in m.

Arguments outside the allowed range are bounded to _min and _max.

Parameters
width – width in m.

Returns
err_type_t::OK

virtual *err_type_t* **getPosition**(float &width)

Gets the gripper width.

Note

Since the PWM servo has no position feedback, the latest position command is returned.

Parameters
width – width in m.

Returns
err_type_t::OK

virtual void **setReduction**(float reduction)

Manually set reduction.

Parameters
reduction –

virtual void **setOffset**(float offset)

Manually set offset.

Private Members

std::chrono::_V2::system_clock::time_point **_new_cmd_time** = std::chrono::high_resolution_clock::now()

class **BaseJoint**

#include <mBaseJoint.h> Generic base class to control a single joint.

This class is a wrapper function to interact with a robot joint either through a *MockJoint* or hardware *Joint* object.

Subclassed by *bioscara_hardware_drivers::Joint*, *bioscara_hardware_drivers::MockJoint*

Public Types

enum **stp_reg_t**

register and command definitions

a register can be read (R) or written (W), each register has a size in bytes. The payload can be split into multiple values or just be a single value. Note that not all functions are implemented.

Values:

enumerator **NONE**

Used for signalling purposes.

enumerator **PING**

R; Size: 1; [(char) ACK].

enumerator **SETUP**

W; Size: 2; [(uint8) holdCurrent, (uint8) driveCurrent].

enumerator **SETRPM**

W; Size: 4; [(float) RPM].

enumerator **GETDRIVERRPM**

enumerator **MOVESTEPS**

W; Size: 4; [(int32) steps].

enumerator **MOVEANGLE**

enumerator **MOVETOANGLE**

W; Size: 4; [(float) degrees].

enumerator **GETMOTORSTATE**

enumerator **RUNCOTINOUS**

enumerator **ANGLEMOVED**

R; Size: 4; [(float) degrees].

enumerator **SETCURRENT**

W; Size: 1; [(uint8) driveCurrent].

enumerator **SETHOLDCURRENT**

W; Size: 1; [(uint8) holdCurrent].

enumerator **SETMAXACCELERATION**

enumerator **SETMAXDECELERATION**

enumerator **SETMAXVELOCITY**

enumerator **ENABLESTALLGUARD**

W; Size: 1; [(uint8) threshold].

enumerator **DISABLESTALLGUARD**

enumerator **CLEARSTALL**

enumerator **SETBRAKEMODE**

W; Size: 1; [(uint8) mode].

enumerator **ENABLEPID**

enumerator **DISABLEPID**

enumerator **ENABLECLOSEDLOOP**

enumerator **DISABLECLOSEDLOOP**

W; Size: 1; [(uint8) 0].

enumerator **SETCONTROLTHRESHOLD**

enumerator **MOVETOEND**

enumerator **STOP**

W; Size: 1; [(uint8) mode].

enumerator **GETPIDERROR**

enumerator **CHECKORIENTATION**

W; Size: 4; [(float) degrees].

enumerator **GETENCODERRPM**

R; Size: 4; [(float) RPM].

enumerator **HOME**

W; Size: 4; [(uint8) current, (int8) sensitivity, (uint8) speed, (uint8) direction].

enumerator **HOMEOFFSET**

R/W; Size: 4; [(float) -].

Public Functions

BaseJoint(const std::string name)

Create a *Joint* object.

The *Joint* object represents a single joint.

Parameters

name – string device name for logging.

~BaseJoint(void)

Destroy the *BaseJoint* object.

Invokes the *disable()* and *deinit()* method to clean up.

virtual *err_type_t* **init**(void)

Initialize the joint communication.

Returns
err_type_t

virtual *err_type_t* **deinit**(void)

Denitialize the joint communication.

Returns
err_type_t

virtual *err_type_t* **enable**(u_int8_t driveCurrent, u_int8_t holdCurrent)

Engages the joint.

This function prepares the motor for movement. After successfull execution the joint is ready to accept *setPosition()* and *setVelocity()* commands.

The function sets the drive and hold current for the specified joint and engages the motor. The currents are in percent of driver max. output (2.5A, check with TMC5130 datasheet or Ustepper documentation)

Parameters

- **driveCurrent** – drive current in 0-100 % of 2.5A output (check uStepper doc.)
- **holdCurrent** – hold current in 0-100 % of 2.5A output (check uStepper doc.)

Returns
err_type_t

virtual *err_type_t* **disable**(void)

disenganges the joint

invokes *stop()*, sets hold and drive current to 0 and sets the the joint brake mode to freewheeling

Returns
err_type_t

virtual *err_type_t* **home**(float velocity, u_int8_t sensitivity, u_int8_t current)

Blocking implementation to home the joint.

Homing the joint is neccessary after its controller has been powered off. The joint can be moved while the encoders are inactive and hence the position at startup is unknow. See _home() for information on implementation and description of the paramters.

This is a blocking implementation which only returns after the the joint is no longer busy. First *startHoming()* is called, and subsequently waits with wait_while_busy() to finish homing. Lastly *postHoming()* is called.

Returns
err_type_t

virtual *err_type_t* **startHoming**(float velocity, u_int8_t sensitivity, u_int8_t current)

non-blocking implementation to home the joint

Homing the joint is neccessary after its controller has been powered off. The joint can be moved while the encoders are inactive and hence the position at startup is unknow. See _home() for information on implementation and description of the paramters.

This method returns immediatly after starting the homing sequence and should be used when the blocking implementation is not acceptable, for example in a realtime loop.

This method sets the current_b_cmd flag to *stp_reg_t::HOME*

Returns
err_type_t

virtual *err_type_t* **postHoming**(void)

perform tasks after a non-blocking homing

This method resets the current_b_cmd to *stp_reg_t::NONE*, checks if the joint is homed, and saves the homing offset to the joint.

Returns
err_type_t

virtual *err_type_t* **getPosition**(float &pos) = 0

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Warning

If the joint is not homed this method does not return an error. Instead `pos` will be 0.0.

Parameters
pos – the current joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **setPosition**(float pos)

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Parameters
pos – the commanded joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **moveSteps**(int32_t steps)

Move full steps.

This function can be called even when not homed.

Parameters
steps – number of full steps

Returns
err_type_t

virtual *err_type_t* **getVelocity**(float &vel) = 0

get the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the current joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **setVelocity**(float vel)

Set the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the commanded joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **checkOrientation**(float angle = 2.0)

Calls the checkOrientation method of the motor. Checks in which direction the motor is turning.

Note

This is a blocking function.

Parameters
angle – degrees how much the motor should turn. A few degrees is sufficient.

Returns
err_type_t

virtual *err_type_t* **stop**(void)

Stops the motor.

Stops the motor by setting the maximum velocity to zero and the position setpoint to the current position

Returns
err_type_t

virtual *err_type_t* **disableCL**(void)

virtual *err_type_t* **setDriveCurrent**(u_int8_t current)

Set the Drive Current.

Parameters
current – 0% - 100% of driver current

Returns
err_type_t

virtual *err_type_t* **setHoldCurrent**(u_int8_t current)

Set the Hold Current.

Parameters
current – 0% - 100% of driver current

Returns
err_type_t -1 on communication error,

virtual *err_type_t* **setBrakeMode**(u_int8_t mode)

Set Brake Mode.

Parameters
mode – Freewheel: 0, Coolbrake: 1, Hardbrake: 2

Returns
err_type_t

virtual *err_type_t* **setMaxAcceleration**(float maxAccel)

Set the maximum permitted joint acceleration (and deceleration) in rad/s^2 or m/s^2 for cylindrical and prismatic joints respectively.

Parameters
maxAccel – maximum joint acceleration.

Returns
err_type_t

virtual *err_type_t* **setMaxVelocity**(float maxVel)

Set the maximum permitted joint velocity in rad/s or m/s for cylindrical and prismatic joints respectively.

Parameters
maxVel – maximum joint velocity.

Returns
err_type_t

virtual *err_type_t* **enableStallguard**(u_int8_t sensitivity)

Enable encoder stall detection of the joint.

If the PID error exceeds the set threshold a stall is triggered and the motor disabled. A detected stall can be reset by homing or reenabling the joint using *enable()*.

Note

If stall detection shall be enabled, invoke this method AFTER enabling the joint with *enable()*.

Parameters
sensitivity – value of threshold. 0 - 255 where lower is more sensitive.

Returns
err_type_t

virtual bool **isHomed**(void)

Checks the state if the motor is homed.

Reads the internal state flags from the last transmission. If an update is neccessary call *getFlags()* before invoking this function.

Returns
true if the motor is homed, false if not.

virtual bool **isEnabled**(void)

Checks the state if the motor is enabled.

Reads the internal state flags from the last transmission. If an update is neccessary call *getFlags()* before invoking this function. If the motor actually can move depends on the state of the STALLED flag which can be checked using *Joint::isStalled()*.

Returns
true if the motor is enabled, false if not.

virtual bool **isStalled**(void)

Checks if the motor is stalled.

Reads the internal state flags from the last transmission. If an update is neccessary call *getFlags()* before invoking this function.

Returns
true if the motor is stalled, false if not.

virtual bool **isBusy**(void)

Checks if the joint controller is busy processing a blocking command.

Reads the internal state flags from the last transmission. If an update is neccessary call *getFlags()* before invoking this function.

Returns
true if a blocking command is currently executing, false if not.

virtual *err_type_t* **getFlags**(u_int8_t &flags)

get the latest driver state flags from the joint

Parameters
flags – if succesfull, populated with the latest flags

Returns
err_type_t

virtual *err_type_t* **getFlags**(void)

Overload of *getFlags(u_int8_t &flags)*

Returns
err_type_t

virtual *stp_reg_t* **getCurrentBCmd**(void)

get the currently active blocking command

Returns
The the command of type stp_reg_t

Public Members

std::string **name**

Joint name for logging.

class **Gripper** : public bioscara_hardware_drivers::*BaseGripper*

#include <mGripper.h> Child class implementing control of the hardware gripper.

Use this class if you wish to control the actual hardware by generating a PWM voltage on GPIO18 of the Raspberry Pi.

Public Functions

Gripper(float reduction, float offset, float min, float max, float backup_init_pos)

Construct a new *Gripper* object.

see the *BaseGripper* constructor for a description of the parameters.

virtual *err_type_t* **enable**(void) override

Activates the the hardware.

Invokes *BaseGripper::enable()* and starts the PWM generation but does not set a position. Must be called before a position is set. The PWM pin is GPIO18 on the Raspberry Pi 4. PWM chip is 0, channel 0.

Returns
err_type_t::OK on success, *err_type_t::COMM_ERROR* if PWM can not be enabled

virtual *err_type_t* **disable**(void) override

Deactivates the hardware.

Invokes *BaseGripper::disable()* and stops the servo by disableing the PWM generation.

Returns
err_type_t::OK

virtual *err_type_t* **setPosition**(float width) override

Sets the gripper width in m.

Invokes *BaseGripper::setPosition()*, transforms the desired gripper width to a servo angle using the JOINT2ACTUATOR() macro and finally sets the servo position using setServoPosition()

Parameters
width – width in m.

Returns
see setServoPosition()

class **Joint** : public bioscara_hardware_drivers::*BaseJoint*

#include <mJoint.h> Representing a single hardware joint connected via I2C.

Public Functions

Joint(const std::string name, const int address, const float reduction, const float min, const float max)

Create a *Joint* object.

The *Joint* object represents a single joint and its actuator. Each *Joint* has a transmission with the following relationship:

actuator position = (joint position - offset) * reduction

joint position = actuator position / reduction + offset

Parameters

- **name** – string device name for identification
- **address** – 1-byte I2C device adress (0x11 … 0x14) for J1 … J4
- **reduction** – gear reduction of the joint. This is used to transform position and velocity values between in joint units and actuator (stepper) units. The sign depends on the direction the motor is mounted and is turning. Adjust such that the joint moves in the positive direction on on positive joint commands. Cable polarity has no effect since the motors automatically adjust to always run in the 'right' direction from their point of view.

  J1: 35

  J2: -2*pi/0.004 (4 mm linear movement per stepper revolution)

  J3: 24

  J4: 12

- **min** – lower joint limit in joint units.

  J1: -3.04647

  J2: -0.0016

  J3: -2.62672

J4: -3.01069

- **max** – upper joint limit in joint units.

  J1: 3.04647

  J2: 0.3380

  J3: 2.62672

  J4: 3.01069

~Joint(void)

Destroy the *Joint* object.

Create a *Joint* object.

virtual *err_type_t* **init**(void) override

Initialize connection to a joint via I2C.

Adds the joint to the I2C bus and tests if is responsive by invoking checkCom(). If the joint is homed, retrieve the homing position stored on the joint by invoking getHomingOffset().

Returns
err_type_t

virtual *err_type_t* **deinit**(void) override

Disconnects from a joint.

Removes the joint from the I2C bus by invoking closeI2CDevHandle().

Returns
err_type_t

virtual *err_type_t* **enable**(u_int8_t driveCurrent, u_int8_t holdCurrent) override

Engages the joint.

This function prepares the motor for movement. After successfull execution the joint is ready to accept *setPosition()* and *setVelocity()* commands.

The function sets the drive and hold current for the specified joint and engages the motor. The currents are in percent of driver max. output (2.5A, check with TMC5130 datasheet or Ustepper documentation)

Parameters

- **driveCurrent** – drive current in 0-100 % of 2.5A output (check uStepper doc.)
- **holdCurrent** – hold current in 0-100 % of 2.5A output (check uStepper doc.)

Returns
err_type_t

virtual *err_type_t* **postHoming**(void) override

perform tasks after a non-blocking homing

Invokes *BaseJoint::postHoming()* and additionally saves the homing offset to the joint controller setHomingOffset()

Returns
err_type_t

virtual *err_type_t* **getPosition**(float &pos) override

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Warning

If the joint is not homed this method does not return an error. Instead `pos` will be 0.0.

Parameters
pos – the current joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **setPosition**(float pos) override

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Parameters
pos – the commanded joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **moveSteps**(int32_t steps) override

Move full steps.

This function can be called even when not homed.

Parameters
steps – number of full steps

Returns
err_type_t

virtual *err_type_t* **getVelocity**(float &vel) override

get the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the current joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **setVelocity**(float vel) override

Set the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the commanded joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **checkOrientation**(float angle = 2.0) override

Calls the checkOrientation method of the motor. Checks in which direction the motor is turning.

Note

This is a blocking function.

Parameters
angle – degrees how much the motor should turn. A few degrees is sufficient.

Returns
err_type_t

virtual *err_type_t* **stop**(void) override

Stops the motor.

Stops the motor by setting the maximum velocity to zero and the position setpoint to the current position

Returns
err_type_t

virtual *err_type_t* **disableCL**(void) override

virtual *err_type_t* **setDriveCurrent**(u_int8_t current) override

Set the Drive Current.

Parameters
current – 0% - 100% of driver current

Returns
err_type_t

virtual *err_type_t* **setHoldCurrent**(u_int8_t current) override

Set the Hold Current.

Parameters
current – 0% - 100% of driver current

Returns
err_type_t -1 on communication error,

virtual *err_type_t* **setBrakeMode**(u_int8_t mode) override

Set Brake Mode.

Parameters
mode – Freewheel: 0, Coolbrake: 1, Hardbrake: 2

Returns
err_type_t

virtual *err_type_t* **setMaxAcceleration**(float maxAccel) override

Set the maximum permitted joint acceleration (and deceleration) in rad/s^2 or m/s^2 for cylindrical and prismatic joints respectively.

Parameters
maxAccel – maximum joint acceleration.

Returns
err_type_t

virtual *err_type_t* **setMaxVelocity**(float maxVel) override

Set the maximum permitted joint velocity in rad/s or m/s for cylindrical and prismatic joints respectively.

Parameters
maxVel – maximum joint velocity.

Returns
err_type_t

virtual *err_type_t* **enableStallguard**(u_int8_t sensitivity) override

Enable encoder stall detection of the joint.

If the PID error exceeds the set threshold a stall is triggered and the motor disabled. A detected stall can be reset by homing or reenabling the joint using *enable()*.

Note

If stall detection shall be enabled, invoke this method AFTER enabling the joint with *enable()*.

Parameters
sensitivity – value of threshold. 0 - 255 where lower is more sensitive.

Returns
err_type_t

virtual *err_type_t* **getFlags**(void) override

Overload of *getFlags(u_int8_t &flags)*

Returns
err_type_t

Private Functions

template<typename **T**>
int **read**(const stp_reg_t reg, *T* &data, u_int8_t &flags)

Wrapper function to request data from the I2C slave.

Allocates a buffer of size sizeof(T) + RFLAGS_SIZE. Invokes readFromI2CDev(), and copies the received payload to *data* and the transmisison flags to *flags*. See BaseJoint::flags for details.

Template Parameters
T – Datatype of value to be transmitted

Parameters

- **reg** – stp_reg_t register to read
- **data** – reference to store payload.
- **flags** – reference to a byte which stores the return flags

Returns
0 on OK, negative on error

template<typename **T**>
int **write**(const stp_reg_t reg, *T* data, u_int8_t &flags)

Wrapper function to send command to the I2C slave.

Allocates a buffer of size sizeof(T) + RFLAGS_SIZE. Copyies *data* to the buffer and invokes writeToI2CDev(). The flags received from the transaction are copied to *flags*. The flags are described in *Joint::read()*.

Template Parameters
T – Datatype of value to be transmitted

Parameters

- **reg** – stp_reg_t command to execute

- **data** – payload to transmit. It is the users responsibility to populate the right amount of data for the relevant register
- **flags** – reference to a byte which stores the return flags

Returns
0 on OK, negative on error

Private Members

int **address**

I2C adress.

int **handle** = -1

I2C bus handle.

class **MockGripper** : public bioscara_hardware_drivers::*BaseGripper*

#include <mMockGripper.h> Child class for to mock the gripper hardware.

Use this class if you dont wish to use the gripper hardware. Commands are simply mirrored.

Public Functions

MockGripper(float reduction, float offset, float min, float max, float backup_init_pos)

Construct a new *MockGripper* object.

see the *BaseGripper* constructor for a description of the parameters.

class **MockJoint** : public bioscara_hardware_drivers::*BaseJoint*

#include <mMockJoint.h> Representing a single joint mocking the hardware joint.

This mock hardware implementation simply mirrors (potentially delayed) commands.

Note

Much of this implementation is very basic not well written.

Public Functions

MockJoint(const std::string name)

virtual *err_type_t* **enable**(u_int8_t driveCurrent, u_int8_t holdCurrent) override

Engages the joint.

This function prepares the motor for movement. After successfull execution the joint is ready to accept *setPosition()* and *setVelocity()* commands.

The function sets the drive and hold current for the specified joint and engages the motor. The currents are in percent of driver max. output (2.5A, check with TMC5130 datasheet or Ustepper documentation)

Parameters

- **driveCurrent** – drive current in 0-100 % of 2.5A output (check uStepper doc.)
- **holdCurrent** – hold current in 0-100 % of 2.5A output (check uStepper doc.)

Returns
err_type_t

virtual *err_type_t* **disable**(void) override

disenganges the joint

invokes *stop()*, sets hold and drive current to 0 and sets the the joint brake mode to freewheeling

Returns
err_type_t

virtual *err_type_t* **getPosition**(float &pos) override

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Warning

If the joint is not homed this method does not return an error. Instead `pos` will be 0.0.

Parameters
pos – the current joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **setPosition**(float pos) override

get the current joint position in radians or m for cylindrical and prismatic joints respectively.

Parameters
pos – the commanded joint position in rad or m.

Returns
err_type_t

virtual *err_type_t* **getVelocity**(float &vel) override

get the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the current joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **setVelocity**(float vel) override

Set the current joint velocity in radians/s or m/s for cylindrical and prismatic joints respectively.

Parameters
vel – the commanded joint velocity in rad/s or m/s.

Returns
err_type_t

virtual *err_type_t* **checkOrientation**(float angle = 2.0) override

Calls the checkOrientation method of the motor. Checks in which direction the motor is turning.

Note

This is a blocking function.

Parameters
angle – degrees how much the motor should turn. A few degrees is sufficient.

Returns
err_type_t

virtual *err_type_t* **stop**(void) override

Stops the motor.

Stops the motor by setting the maximum velocity to zero and the position setpoint to the current position

Returns
err_type_t

virtual *err_type_t* **getFlags**(void) override

Overload of *getFlags(u_int8_t &flags)*

Returns
err_type_t

virtual bool **isHomed**(void) override

Checks the state if the motor is homed.

Reads the internal state flags from the last transmission. If an update is neccessary call *getFlags()* before invoking this function.

Returns
true if the motor is homed, false if not.

Private Functions

float **getDeltaT**(std::chrono::_V2::system_clock::time_point &last_call, bool update = true)

Compute the time since the given time point.

Parameters

- **last_call** – the time point to calculate the duration
- **update** – If true compute the duration and also reset *last_call* to current time

Returns
elapsed time in seconds since *last_call*

Private Members

float **q** = 0.0

position

float **qd** = 0.0

velocity

std::chrono::_V2::system_clock::time_point **last_set_position** =
std::chrono::high_resolution_clock::now()

std::chrono::_V2::system_clock::time_point **last_set_velocity** = *last_set_position*

std::chrono::_V2::system_clock::time_point **async_start_time** = *last_set_position*

stp_reg_t **op_mode** = NONE

INDEX

B